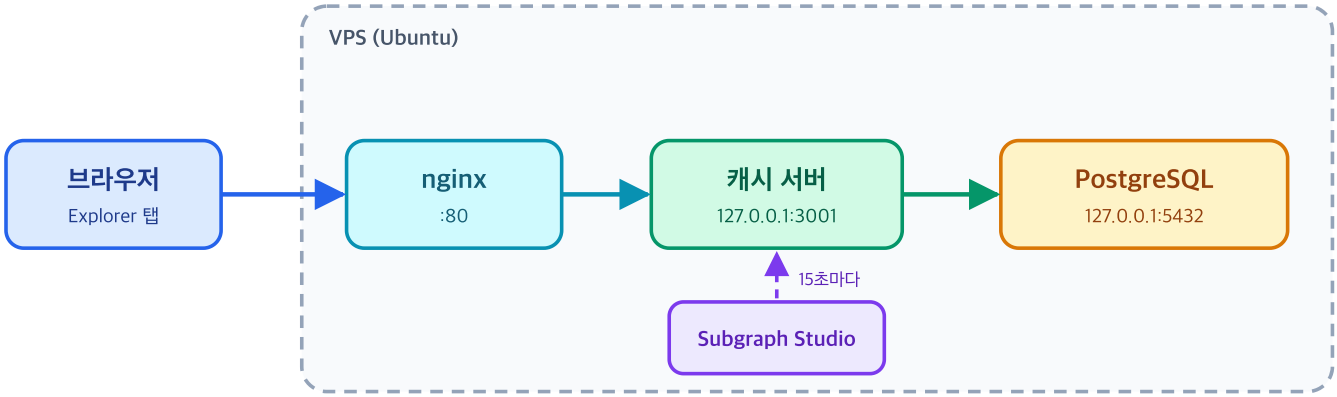
TOKENZSWAP 학습 가이드

캐시 서버와 서버 배포

PostgreSQL 준비부터 캐시 서버, Explorer, nginx, systemd, 방화벽까지



목차

1. 전체 그림
2. 왜 캐시 서버인가
3. PostgreSQL 준비
4. SSH 터널 (개발용 통로)
5. db.ts: 테이블과 책갈피
6. sync.ts: 바뀐 것만 가져오기
7. server.ts: JSON API
8. index.ts: 시작 순서와 15초 루프
9. Explorer: 프론트 연결
10. 서버 안의 지도
11. 코드 올리기와 .env
12. systemd: 24시간 돌리기
13. nginx: 입구 하나로 나누기
14. 방화벽과 fail2ban
15. 업데이트 흐름 (자세히)
16. 문제 해결 사례와 체크리스트

1. 전체 그림

이 가이드가 다루는 부분은 **"서브그래프 데이터를 내 서버에 복사해 두고, 그걸 웹페이지에 보여 주는 것"**입니다. 먼저 모든 부품이 어디에 있고 무엇과 연결되는지 한 장으로 봅니다.

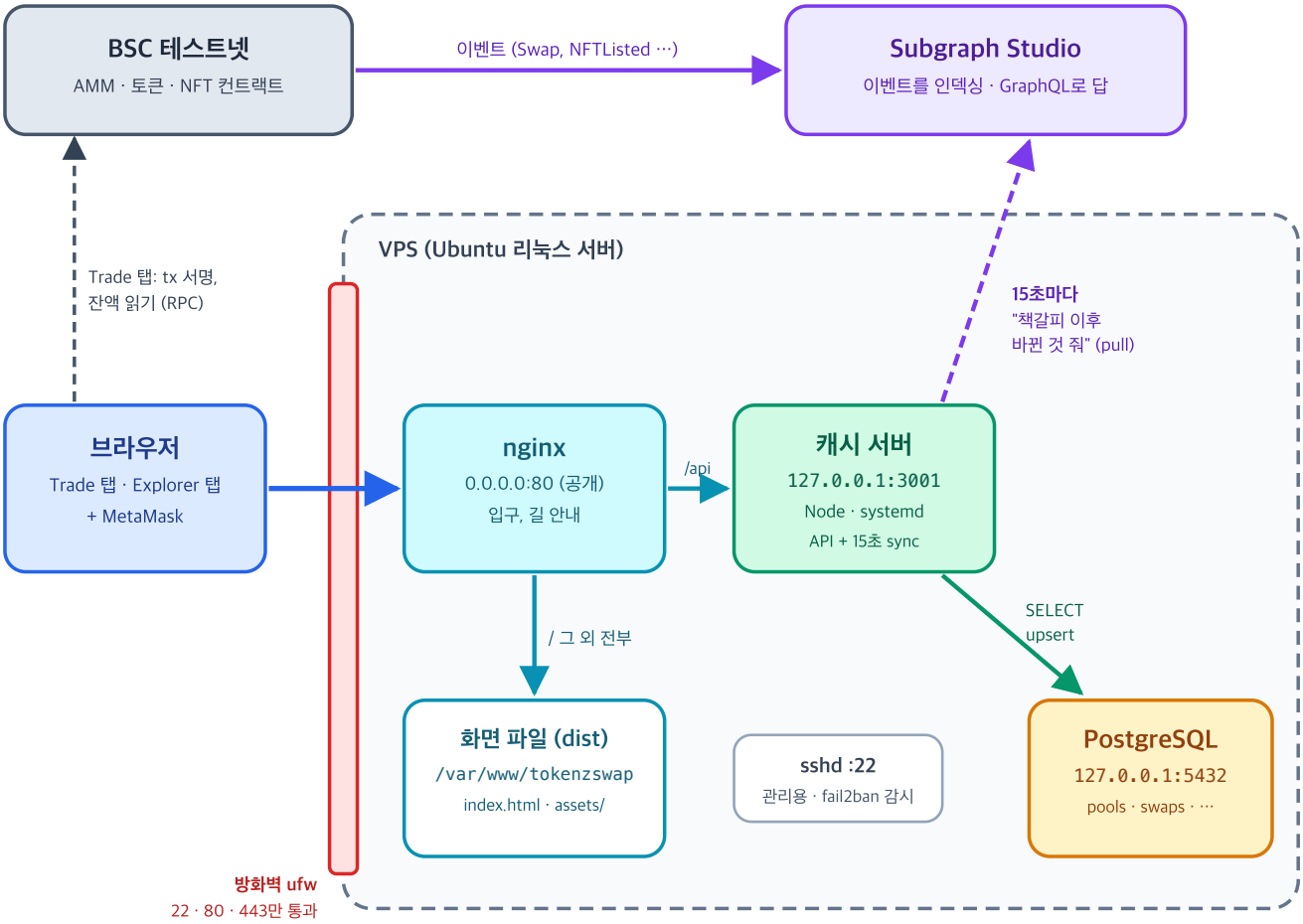


그림 1. 전체 구조. 밖에서 들어올 수 있는 문은 nginx(80)와 SSH(22)뿐이고, 캐시 서버와 DB는 서버 안(127.0.0.1)에만 열려 있습니다.

부품	한 문장 정의	어디서
Subgraph Studio	체인 이벤트를 읽어 정리해 두고, 묻는 말에 GraphQL로 답하는 서버	인터넷 (The Graph)
캐시 서버	15초마다 Studio에서 바뀐 것만 가져와 DB에 넣고, 브라우저에는 DB 내용을 JSON으로 주는 프로그램	VPS, 127.0.0.1:3001
PostgreSQL	캐시한 데이터를 표(테이블)로 저장하는 데이터베이스	VPS, 127.0.0.1:5432
nginx	인터넷에서 온 요청을 받아 "/api는 캐시 서버로, 나머지는 화면 파일로" 나눠 주는 입구	VPS, 0.0.0.0:80
브라우저	Explorer 탭은 /api만 부르고, Trade 탭은 MetaMask로 체인과 직접 통신	사용자 컴퓨터

1-2. swap 한 번이 화면에 나오기까지

실제로 해 본 것: 프론트에서 1 SW42를 swap했더니, 1분 안에 캐시 서버 로그에 `sync 139637387 → 139637426: 2 rows`가 찍히고 Explorer의 swap Count가 2로 바뀌었습니다. 그 사이에 일어난 일을 시간 순서대로 그리면 이렇습니다.

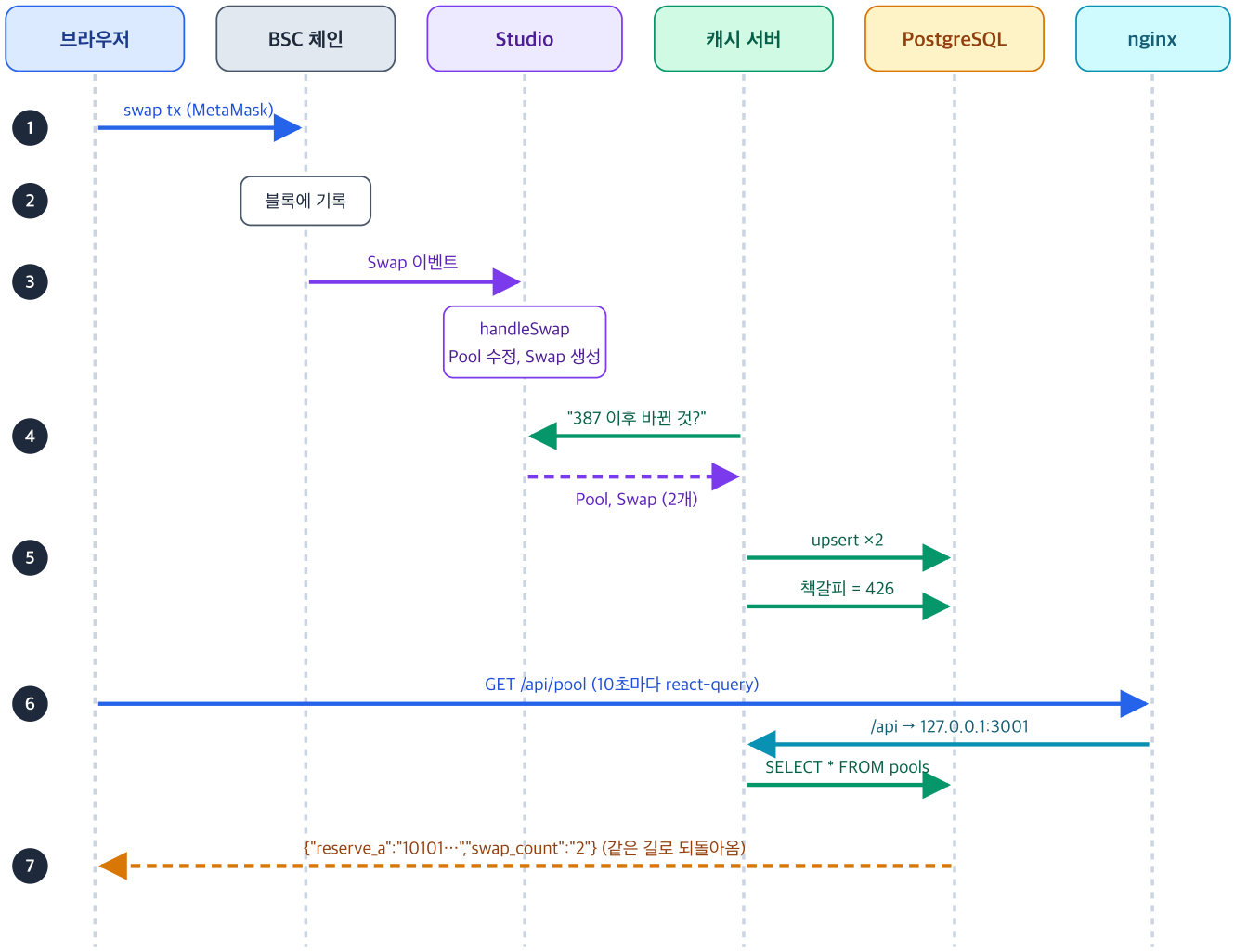


그림 2. swap 한 번의 여행. 1~3은 체인과 The Graph, 4~5는 캐시 서버의 15초 루프, 6~7은 브라우저의 요청입니다.

#	누가	무엇을	걸리는 시간
1-2	MetaMask → BSC	tx가 블록에 들어감	수 초
3	The Graph	이벤트를 읽어 Pool, Swap 엔티티를 고치고 만듦	보통 수십 초
4-5	캐시 서버	다음 15초 바퀴에서 바뀐 2개를 받아 DB에 upsert, 책갈피 이동	최대 15초
6-7	브라우저	10초마다 /api/pool을 다시 물어봄 → 새 값 표시	최대 10초

> 중요한 방향: **Studio가 우리에게 보내 주는 게 아니라, 캐시 서버가 15초마다 가서 가져옵니다 (pull).** Studio는 우리 DB가 있는지도 모릅니다. 캐시 서버가 꺼져 있으면 DB는 멈춰 있다가, 다시 켜면 책갈피부터 따라잡습니다.

2. 왜 캐시 서버인가

Explorer 탭은 처음에 브라우저가 Studio에 직접 GraphQL을 보냈습니다. 그걸 "내 서버에 두는 방식"으로 바꾼 이유입니다.



그림 3. 방문자가 늘어도 Studio에 가는 요청은 15초에 한 번으로 그대로입니다.

	브라우저가 Studio에 직접	캐시 서버
Studio 요청 수	방문자 수 × 새로고침 수	15초에 1번 (방문자 수와 무관)
데이터가 있는 곳	Studio에만	내 PostgreSQL에도 사본 → SQL로 마음대로 조회
쿼리가 있는 곳	프론트 코드 (누구나 봄)	서버 (프론트는 주소만 부름)
Studio가 잠깐 멈추면	화면도 멈춤	마지막 데이터를 계속 보여 줌
대가	없음	서버를 직접 운영 (이 가이드의 나머지 전부)

데이터는 어디에 몇 벌 있나

곳	무엇	원본인가
BSC 체인	컨트랙트 상태와 이벤트 로그	**원본** (진짜)
Studio (graph-node의 Postgres)	이벤트를 정리한 엔티티	체인에서 계산한 사본
내 PostgreSQL	Studio 엔티티를 복사한 테이블	Studio의 사본 (캐시)

캐시는 언제든 지우고 다시 만들 수 있습니다. 책갈피를 0으로 되돌리면 처음부터 다시 받아 옵니다.

3. PostgreSQL 준비

> PostgreSQL은 "데이터를 표(테이블)로 저장하고, SQL이라는 질문 언어로 꺼내 쓰게 해 주는 서버 프로그램"입니다.

3-1. 세 가지 이름: 서버, 계정(role), 데이터베이스



그림 4. 서버 하나 안에 계정과 데이터베이스가 있고, 데이터베이스 안에 테이블이 있습니다.

3-2. 설치와 계정 만들기 (VPS에서 한 번)

```
sudo apt install -y postgresql                    # 설치하면 systemd 서비스로 자동 시작
sudo -u postgres createuser tokenzswap            # 앱 전용 계정
sudo -u postgres createdb -O tokenzswap tokenzswap # 그 계정이 주인인 DB
sudo -u postgres psql -c "\password tokenzswap"    # 비밀번호 설정 (화면·기록에 안 남음)
```

명령	뜻
`sudo -u postgres …`	리눅스의 postgres 사용자로 실행. 설치 직후에는 이 사용자만 DB 관리자로 들어갈 수 있음
`createdb -O tokenzswap`	-O = owner. 앱 계정이 자기 DB 안에서 테이블을 만들 수 있게
`\password`	`ALTER ROLE … PASSWORD '…'` 처럼 비밀번호를 명령에 쓰면 셸 기록에 남음. `\password` 는 입력을 숨김

3-3. 접속 확인: bash 프롬프트 vs psql 프롬프트

프롬프트	지금 있는 곳	쓸 수 있는 것
`ubuntu@vod-sun-…:~$`	bash (리눅스 셸)	`ls` , `sudo` , `psql` 같은 명령
`tokenzswap=>`	psql (DB에 접속한 상태)	SQL (`SELECT … ;`), psql 명령 (`\dt` , `\q`)

```
psql -h localhost -U tokenzswap -d tokenzswap -W   # bash에서: 접속 (-W = 비밀번호 물어보기)
tokenzswap=> \dt                                   # 테이블 목록
tokenzswap=> SELECT * FROM sync_state;             # 세미콜론 필수
tokenzswap=> \q                                    # 나가기
```

> 실제로 겪은 일: bash 프롬프트에 `select * from sync_state` 를 쳐서 `syntax error near unexpected token 'from'` , `\dt` 를 쳐서 `dt: command not found` 가 났습니다. SQL은 psql 안에서만 통합니다.

3-4. 127.0.0.1에서만 듣게 (가장 중요)

```
sudo ss -ltnp | grep 5432
LISTEN 0  200  127.0.0.1:5432  0.0.0.0:*  users:(("postgres",…))
```

Local Address	누가 접속 가능	판정
`127.0.0.1:5432`	이 서버 안의 프로그램만	✅ 우리 설정 (Ubuntu 기본값)
`0.0.0.0:5432` 또는 공인 IP	인터넷 누구나 (비밀번호 추측 공격 대상)	❌

4. SSH 터널: 개발 중에만 쓰는 통로

> SSH 터널(-L)은 "내 컴퓨터의 문(15432)으로 들어온 것을 SSH 연결을 타고 서버 안의 5432로 보내 주는 통로"입니다.

개발할 때 캐시 서버는 **맥**에서 돌았고, DB는 **VPS 안 127.0.0.1**에만 열려 있었습니다. 밖에서 직접 5432로는 못 들어가니, 이미 열려 있는 SSH(22)를 통로로 빌렸습니다.

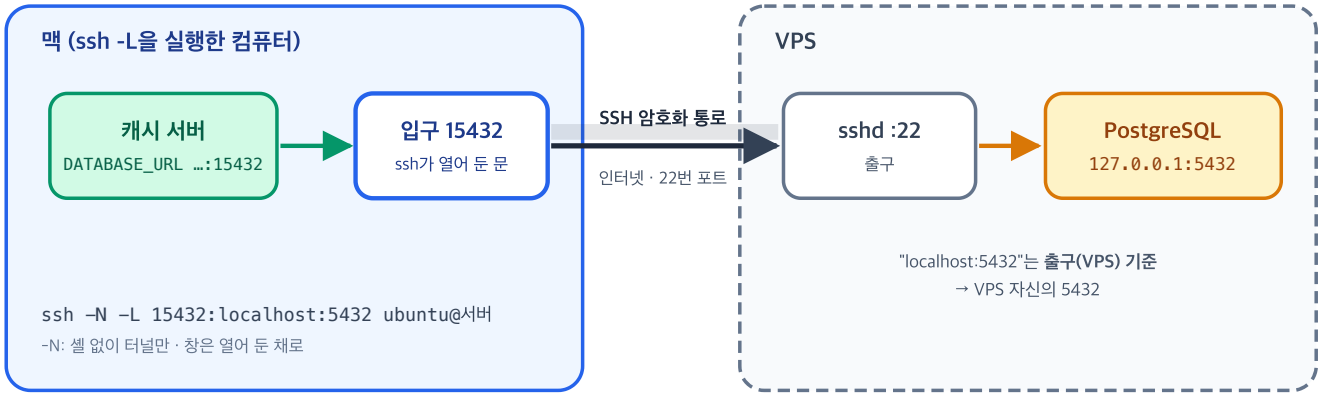


그림 5. `-L 15432:localhost:5432` = "내 15432로 들어오면, 저쪽에서 localhost:5432로 보내라".

-L 15432:localhost:5432 조각	뜻
`15432`	**ssh를 실행한 컴퓨터**에 열리는 입구 번호
`localhost:5432`	SSH 반대편(VPS)에서 바라본 목적지. VPS 자기 자신의 5432

실제로 막혔던 세 가지

증상	원인	교훈
`Operation timed out`	IP 주소를 잘못 씀	timeout = 상대가 아예 대답 안 함 (꺼짐/IP 틀림/방화벽). refused = 켜졌지만 거절. Permission denied = 연결은 됨
VPS에서 `pg_isready -p 15432`는 되는데 맥 캐시 서버는 `ECONNREFUSED`	터널 명령을 **VPS 안**에서 실행 → 입구가 VPS에 생김	입구는 ssh를 친 컴퓨터에 생긴다. 확인도 그 컴퓨터에서
맥에 `ssh ubuntu@…`는 떠 있는데 15432가 비어 있음	`-L` 없이 그냥 로그인만 함	터널 창은 비밀번호 입력 후 **프롬프트 없이 멈춰 있어야** 정상

> 배포 후에는 캐시 서버도 VPS 안에서 돌기 때문에 터널이 필요 없습니다. VPS의 `.env`는 `localhost:5432`로 바로 접속합니다 (11장).

5. db.ts: 테이블과 책갈피

부품	한 문장 정의
`pool`	DB 서버와의 연결을 여러 개 만들어 두고 돌려 쓰는 연결 묶음 (주소는 `DATABASE_URL`에서)
`initDb()`	테이블 6개를 만드는 함수. `IF NOT EXISTS`라서 몇 번을 불러도 안전
`getLastBlock()`	"지난번에 어느 블록까지 가져왔는지" 읽는 책갈피 함수 (없으면 0)
`setLastBlock(n)`	책갈피를 n으로 저장하는 함수 (없으면 INSERT, 있으면 UPDATE)

칼럼 타입을 이렇게 정한 이유

값	타입	이유
reserve, amount, price, liquidity	`NUMERIC(78,0)`	uint256 최댓값은 78자리. 소수점 0자리로 wei를 그대로 저장. JS의 number(약 16자리 정밀도)로는 깨짐 → pg 드라이버가 **문자열**로 돌려줌
timestamp, swap_count	`BIGINT`	정수지만 int(21억)보다 클 수 있음
주소, id, status	`TEXT`	글자
buyer	`TEXT` (NOT NULL 없음)	팔리기 전에는 값이 없음 → NULL 허용

칼럼 이름이 `reserve_a`처럼 snake_case인 이유: PostgreSQL은 따옴표 없는 이름을 **소문자로 바꿔 버립니다**. `reserveA`로 만들면 `reservea`가 되어 읽기 어렵습니다.

값 자리표시 $1, $2 와 upsert

```
await pool.query(
  `INSERT INTO sync_state (key, value) VALUES ($1, $2)
   ON CONFLICT (key) DO UPDATE SET value = EXCLUDED.value`,
  ['lastBlock', String(n)],
)
```

조각	뜻
`$1, $2`	값이 들어갈 자리. 실제 값은 뒤의 배열에서 순서대로. SQL 문자열에 값을 직접 붙이지 않음 (7장 SQL 인젝션)
`ON CONFLICT (key)`	같은 key가 이미 있으면 에러 대신…
`DO UPDATE SET value = EXCLUDED.value`	…값을 고친다. `EXCLUDED` = "넣으려다 막힌 새 값"
`await`	DB는 네트워크 너머라 Promise를 줌. 기다리지 않으면 저장 전에 함수가 끝남

> 확인했던 결과: `getLastBlock()` → 0, `setLastBlock(123)` 후 → 123. 이 두 숫자로 "쓰기와 읽기가 VPS DB까지 간다"를 증명했습니다.

6. sync.ts: 바뀐 것만 가져오기

순서	부품	한 문장 정의	왜 필요한가
1	`gql()`	서브그래프에 질문 하나 보내고 답을 받아 오는 함수	fetch·에러 검사를 매번 반복하지 않으려고
2	`ENTITIES`	"서브그래프의 이 필드를 DB의 이 칼럼에 넣어라"를 적은 표	`amountIn` ↔ `amount_in` 번역, 엔티티 5종을 코드 하나로
3	`upsert()`	없으면 넣고, 이미 있으면 새 값으로 고치는 함수	listing이 ACTIVE → SOLD로 바뀌어 다시 오기 때문
4	`syncOnce()`	책갈피 이후에 바뀐 것만 가져와서 DB에 넣는 함수	매번 전부 받으면 느리고 Studio 사용량 낭비

6-1. 책갈피가 움직이는 모습

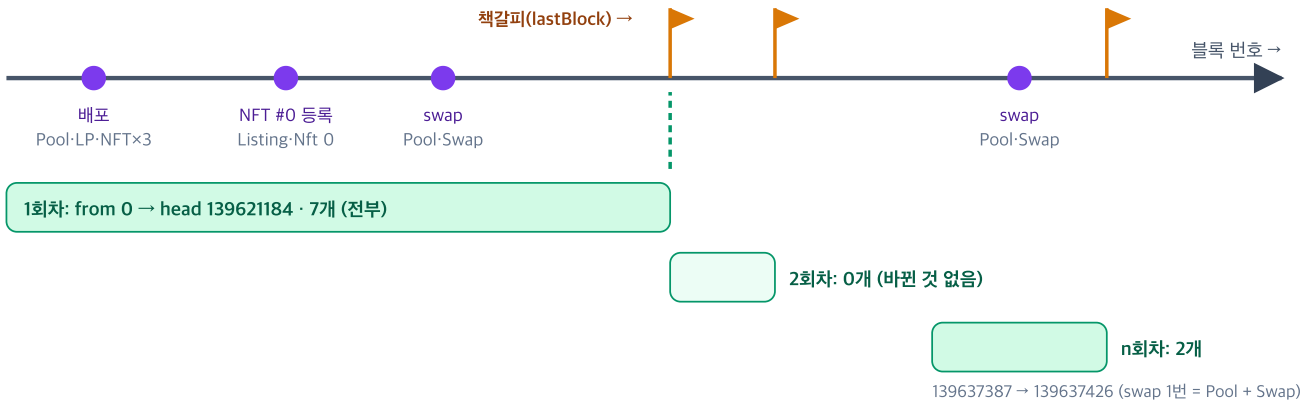


그림 6. 초록 막대는 한 번의 syncOnce가 훑는 구간(from ~ head). 실행이 끝날 때마다 주황 깃발(책갈피)이 head로 옮겨집니다.

6-2. syncOnce 한 바퀴

```
const from = await getLastBlock()                          // ① 책갈피
const head = (await gql('{ _meta { block { number } } }'))._meta.block.number   // ②
for (const e of ENTITIES) {                                // ③ 엔티티 5종 각각
  let cursor = ''
  for (;;) {
    rows = e.field(first: 1000, orderBy: id, block: { number: $head },
             where: { _change_block: { number_gte: $from }, id_gt: $cursor })
    for (const r of rows) await upsert(e.table, e.columns, r)
    if (rows.length < 1000) break                          // 마지막 페이지
    cursor = rows[rows.length - 1].id                        // 다음 페이지는 이 id 다음부터
  }
}
await setLastBlock(head)                                   // ④ 모두 저장한 뒤에만 책갈피 이동
```

필터	뜻	없으면?
`_change_block: { number_gte: from }`	from 이후에 **생기거나 바뀐** 엔티티만	매번 전부 받음
`block: { number: head }`	모든 페이지를 head 시점으로 고정	받는 도중 새 블록이 섞임
`id_gt: cursor` + `first: 1000`	한 번에 최대 1000개, 다음은 마지막 id 다음부터	1000개 넘으면 잘림

6-3. 한 행 넣기: 이름표는 c, 값은 f

```
const cols = columns.map(([, c]) => c).join(', ')        // DB 칼럼 이름  → c (reserve_a)
const vals = columns.map(([f]) => row[f] ?? null)       // 서브그래프 값 → f (reserveA)
`INSERT INTO ${table} (${cols}) VALUES ($1, $2, …) ON CONFLICT (id) DO UPDATE SET reserve_a =
EXCLUDED.reserve_a, …`
```

실제로 겪은 버그 두 개
① `row[c]` 로 써서 `token_a`, `reserve_a` 가 전부 null → NOT NULL 위반. row는 서브그래프에서 왔으니 키가 camelCase(`f`).
② GraphQL 문자열(백틱) 안에 `// 주석` 을 넣어서 `Unexpected character '/'`. 백틱 안은 코드가 아니라 서버로 보내는 글자. GraphQL 주석은 `#`.

체인에서 일어난 일	다음 sync에서 받는 것
swap	Pool (수정) + Swap (새로) = 2
유동성 공급/회수	Pool + LiquidityEvent = 2
NFT 등록	Listing (새로) + Nft (주인 → AMM) = 2
NFT 구매	Listing (SOLD) + Nft (주인 → 구매자) = 2

7. server.ts: JSON API

> API 서버는 "주소로 요청이 오면 맞는 SELECT를 실행하고 결과를 JSON으로 답하는 심부름꾼"입니다. 브라우저는 DB에 직접 붙을 수 없고, 붙어서도 안 되기 때문입니다.

부품	한 문장 정의
`sendJson()`	상태 코드 + `Content-Type: application/json` + `JSON.stringify(body)` 로 답장 하나 보내는 함수
`routes`	"이 주소로 요청이 오면 이 SQL을 실행해라"를 적은 표
`startServer()`	요청을 받아 routes에서 찾아 실행하고 답하는 서버를 127.0.0.1에서 켜는 함수

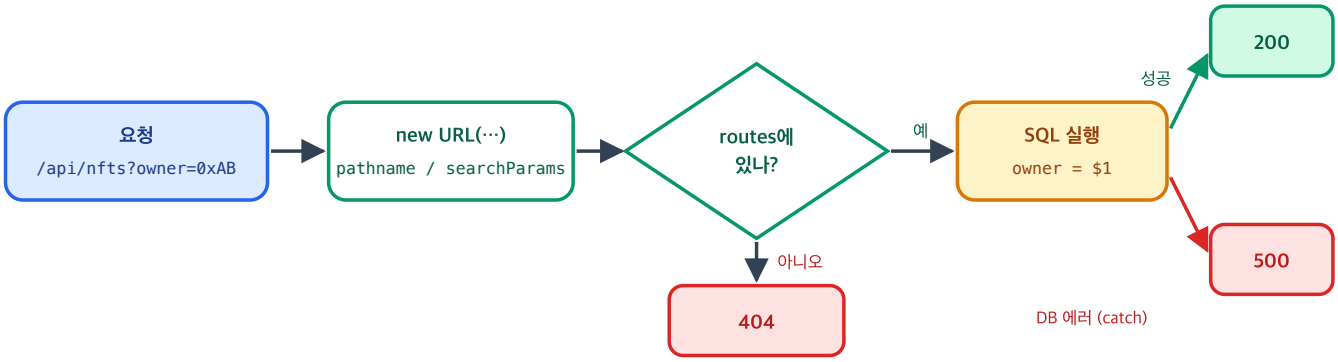


그림 7. 요청 하나의 처리. 500일 때 브라우저에는 "internal error"만, 자세한 내용은 서버 로그에만 남깁니다.

주소	SQL	답
`/api/pool`	`SELECT * FROM pools`	객체 하나 (`rows[0] ?? null`)
`/api/swaps`	`… ORDER BY timestamp DESC LIMIT 20`	최근 20개
`/api/liquidity`	`… ORDER BY timestamp DESC LIMIT 20`	최근 20개
`/api/listings?status=`	`… WHERE status = $1`	상태별
`/api/nfts?owner=`	`… WHERE owner = $1 ORDER BY id::numeric`	주소는 `toLowerCase()` , 번호는 숫자 순

SQL 인젝션: 바깥 값은 무조건 $1

누가 보낸 owner	`'${owner}'` 로 붙이면	`$1` 로 넘기면 (실제 결과)
`0x8124…`	정상	#1, #2
`x' OR '1'='1`	조건이 항상 참 → **전부** 나옴	`[]`
`x'; DROP TABLE nfts; --`	**테이블 삭제**	`[]` , 테이블 그대로

`$1` 로 넘긴 값은 PostgreSQL이 SQL로 해석하지 않고 **글자 값**으로만 비교합니다. `server.listen(port, '127.0.0.1')` 도 같은 철학입니다: 캐시 서버는 nginx를 거쳐서만 닿습니다.

8. index.ts: 시작 순서와 15초 루프

```
await initDb()        // ① 테이블이 있어야 SELECT/INSERT가 됨
startServer(PORT)     // ② 이제부터 요청을 받음
loop()                // ③ sync 시작, 이후 15초마다

async function loop() {
  try {
    const r = await syncOnce()
    if (r.changed > 0) console.log(`sync ${r.from} → ${r.head}: ${r.changed} rows`)
  } catch (err) { console.error('sync failed:', err) }   // 죽지 않고 다음 바퀴에 재시도
  setTimeout(loop, 15_000)                                // 함수 이름만! loop()가 아님
}
```



그림 8. setTimeout을 "끝난 뒤에" 걸면 두 바퀴가 절대 겹치지 않습니다.

코드	뜻	결과
`setTimeout(loop, 15000)`	15초 뒤에 loop를 불러 줘 (함수를 맡김)	✅ 15초 간격
`setTimeout(loop(), 15000)`	지금 당장 loop를 부르고 그 결과를 맡김	❌ 쉬지 않고 Studio에 요청 폭탄

> 화면에 아무것도 안 뜨는 게 정상입니다. 바뀐 게 없으면(changed 0) 로그를 찍지 않습니다. 확인하려면 책갈피를 두 번 읽어 보면 됩니다: 139637140 → 16초 뒤 139637206 (혼자 움직임).

9. Explorer: 프론트 연결

> api()는 "내 캐시 서버에 GET 요청 하나 보내고 JSON을 받아 오는 함수"입니다. 프론트의 gql()을 대신합니다.

```
export async function api<T>(path: string): Promise<T> {
  const res = await fetch(`/api${path}`)           // 옵션 없으면 GET
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  return (await res.json()) as T
}
```

개발 때와 배포 때, 같은 코드가 도는 이유

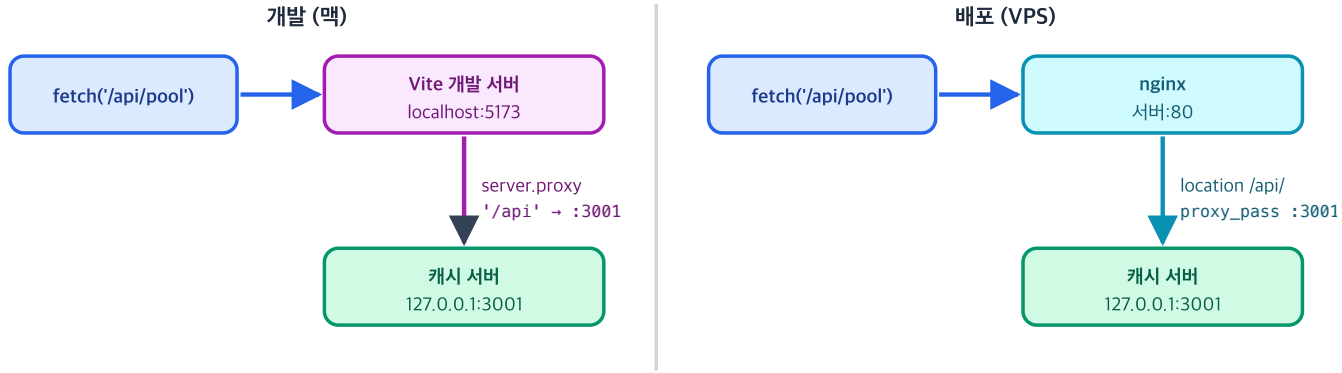


그림 9. 프론트는 항상 같은 사이트의 `/api`만 부릅니다. 개발 때는 Vite가, 배포 때는 nginx가 그걸 3001로 넘깁니다. 같은 사이트라 CORS 문제도 없습니다.

컴포넌트	API	포인트
PoolStats	`/pool`	객체 하나 → `pool.data.reserve_a` (예전: `pools[0].reserveA`)
History	`/swaps` + `/liquidity`	`Promise.all`로 동시에 (100ms+100ms가 아니라 100ms)
MarketHistory	`/listings?status=`	**queryKey에 filter**를 넣어야 버튼마다 새로 요청
NftOwners	`/nfts?owner=`	input(치는 중)과 owner(검색함) 분리, `encodeURIComponent`

값을 지키는 곳	도구	막는 것
URL 안 (프론트)	`encodeURIComponent`	`&`, `#`, `=` 때문에 값이 쪼개지거나 잘림
SQL 안 (서버)	`$1`	SQL 인젝션

10. 서버 안의 지도

여기부터는 배포입니다. 먼저 배포가 끝난 VPS 안에 **무엇이, 어느 폴더에, 누구 권한으로, 몇 번 포트에** 있는지 한 장으로 봅니다.

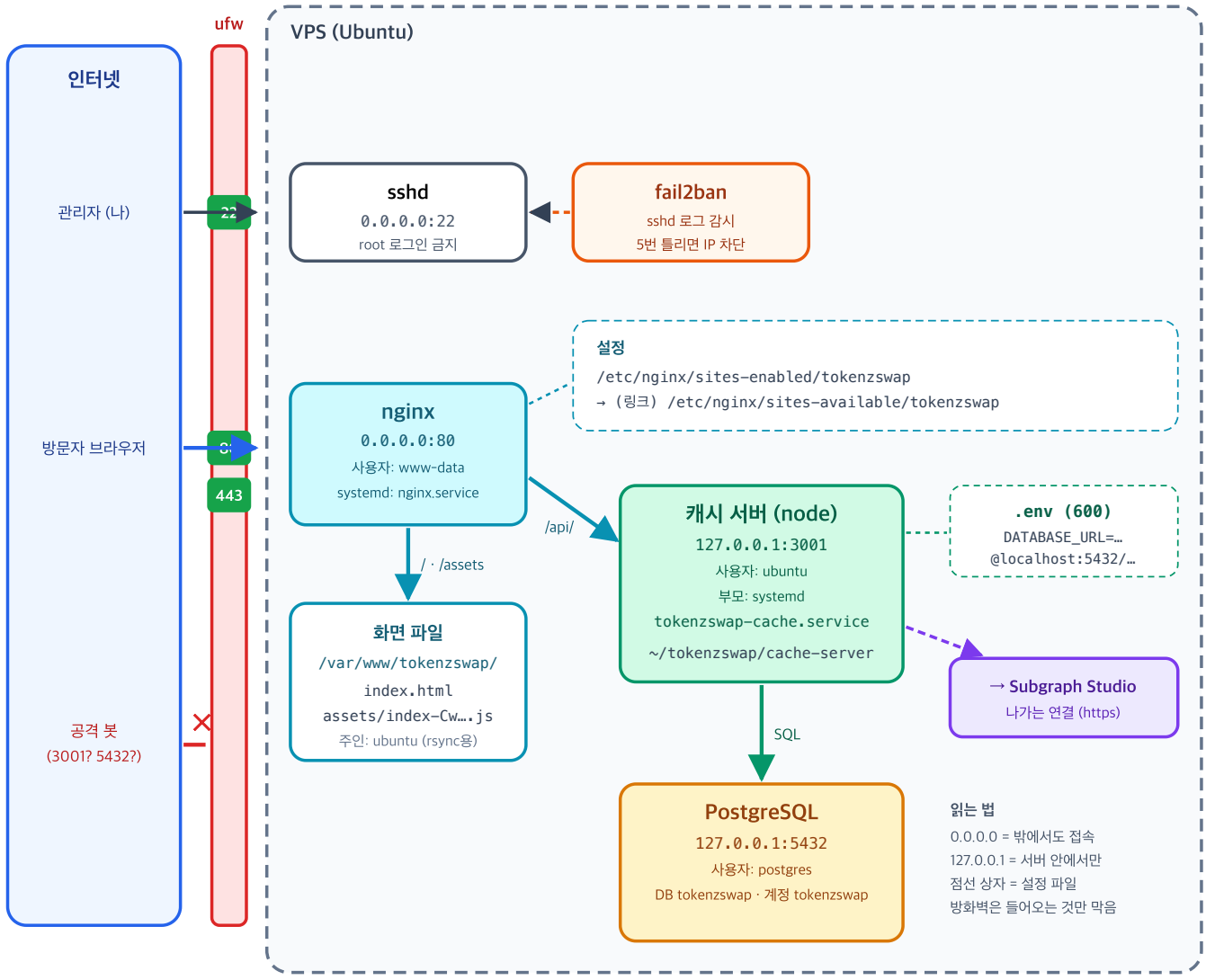


그림 10. 배포가 끝난 VPS. 공격 봇이 3001이나 5432를 두드려도 방화벽에서 막히고, 막히지 않더라도 두 프로그램은 127.0.0.1에서만 듣기 때문에 닿지 않습니다 (이중 잠금).

프로그램	듣는 곳	실행 계정	누가 켜고 살리나	파일 위치
sshd	0.0.0.0:22	root	systemd (ssh.service)	/etc/ssh/sshd_config.d/10-hardening.conf
nginx	0.0.0.0:80	www-data	systemd (nginx.service)	/etc/nginx/sites-available/tokenzswap
캐시 서버	127.0.0.1:3001	ubuntu	systemd (tokenzswap-cache.service)	~/tokenzswap/cache-server
PostgreSQL	127.0.0.1:5432	postgres	systemd (postgresql.service)	/var/lib/postgresql
fail2ban	(포트 없음)	root	systemd (fail2ban.service)	기본 설정 사용

모든 것을 systemd가 관리합니다. 그래서 VPS를 재부팅해도 전부 저절로 다시 켜집니다.

11. 코드 올리기와 .env

11-1. Node 설치

```
sudo apt update && sudo apt install -y nodejs npm
node -v          # 20.6 이상이어야 --env-file 사용 가능
```

11-2. rsync로 보내기

> rsync는 "맥의 폴더를 서버 폴더와 똑같이 맞춰 주는 복사 명령"입니다. 두 번째부터는 바뀐 파일만 보냅니다.

```
# VPS (한 번): 받을 폴더 만들기. rsync는 마지막 폴더 하나만 만들어 줌
mkdir -p ~/tokenzswap/cache-server

# 맥, 프로젝트 루트에서
rsync -av --exclude node_modules --exclude .env \
  code/cache-server/  ubuntu@서버:~/tokenzswap/cache-server/
```

조각	뜻
`-a` / `-v`	폴더 구조·권한 그대로 / 보내는 파일 이름 표시
`--exclude node_modules`	무겁고, 서버에서 `npm ci`로 새로 설치하는 게 맞음
`--exclude .env`	비밀번호가 들어 있고, 서버용은 내용이 다름
`cache-server/` 끝의 /	폴더 **안의 내용**을 보냄 (빼면 cache-server/cache-server/가 생김)

실제로 겪은 일: `mkdir "/home/ubuntu/tokenzswap/cache-server" failed: No such file or directory`. 부모 폴더 `tokenzswap`가 없었습니다. `ssh ubuntu@서버 'mkdir -p ~/tokenzswap/cache-server'`처럼 맥에서 한 줄로 만들면 "어느 창에서 쳤나" 헷갈릴 일이 없습니다.

11-3. npm ci와 서버용 .env

```
cd ~/tokenzswap/cache-server
npm ci                      # package-lock.json의 정확한 버전 그대로 설치
nano .env                   # DATABASE_URL=postgres://tokenzswap:비밀번호@localhost:5432/tokenzswap
chmod 600 .env              # 나(ubuntu)만 읽고 쓰기 → -rw-------
```

맥의 .env (개발)

…tokenzswap:•••@localhost:15432/…

캐시 서버 → 맥의 터널 입구 15432
→ SSH → VPS의 5432

터널 창이 켜져 있어야 동작

VPS의 .env (배포)

…tokenzswap:•••@localhost:5432/…

캐시 서버와 PostgreSQL이 같은 컴퓨터
→ 바로 접속, 터널 불필요

비밀번호는 같은 값 (같은 DB 계정)

그림 11. 두 .env의 차이는 포트 하나뿐입니다.

DB 비밀번호 (.env)	VPS 로그인 비밀번호 (ssh, rsync)
PostgreSQL의 `tokenzswap` 계정, `\password` 로 정함	리눅스의 `ubuntu` 계정
잊으면 `sudo -u postgres psql -c "\password tokenzswap"` 로 새로 정하고 두 .env 모두 수정	잊으면 클라우드 관리 콘솔에서 재설정

12. systemd: 24시간 돌리기

> systemd 서비스 파일은 "이 프로그램을 이 폴더에서 이 명령으로 켜고, 죽으면 다시 켜고, 부팅할 때 자동으로 켜라"를 적은 설정 파일입니다.

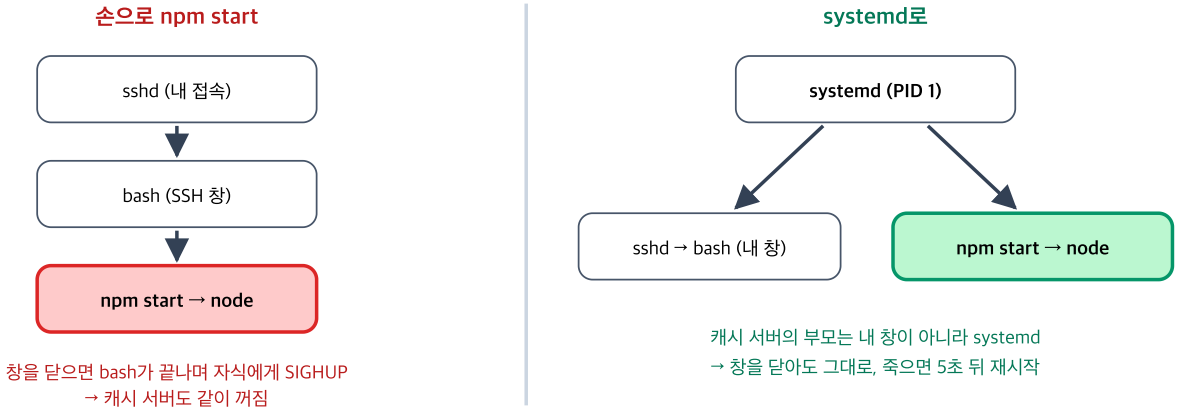


그림 12. 프로세스 가계도. 누가 부모인지가 "창을 닫으면 꺼지는가"를 결정합니다.

```
# /etc/systemd/system/tokenzswap-cache.service  (저장소: deployment/server/)
[Unit]
Description=TokenZswap cache server
After=network-online.target postgresql.service     # 네트워크와 DB 다음에 켜기
Wants=network-online.target

[Service]
User=ubuntu                                        # root로 돌리지 않음
WorkingDirectory=/home/ubuntu/tokenzswap/cache-server   # 여기서 package.json, .env를 찾음
ExecStart=/usr/bin/npm start                       # = tsx --env-file=.env src/index.ts
Restart=always                                     # 죽으면 다시 켜기
RestartSec=5                                       # 5초 뒤에 (무한 재시작 폭주 방지)

[Install]
WantedBy=multi-user.target                         # 부팅 때 자동 시작
```

```
sudo systemctl daemon-reload                  # 새 파일 읽기
sudo systemctl enable --now tokenzswap-cache  # 부팅 등록(enable) + 지금 켜기(--now)
systemctl status tokenzswap-cache --no-pager  # Active: active (running)
journalctl -u tokenzswap-cache -f             # console.log가 쌓이는 곳 (-f: 실시간)
```

명령	언제
`systemctl restart tokenzswap-cache`	코드를 새로 올린 뒤
`systemctl stop / start`	잠깐 끄기 / 켜기
`systemctl disable`	부팅 자동 시작 해제
`daemon-reload`	.service 파일을 고친 뒤 (반드시)

13. nginx: 입구 하나로 나누기

> nginx 설정은 "/api로 시작하는 요청은 캐시 서버로 넘기고, 나머지는 화면 파일(dist)로 답하라"를 적은 표입니다. 개발 때 Vite 프록시가 하던 일입니다.

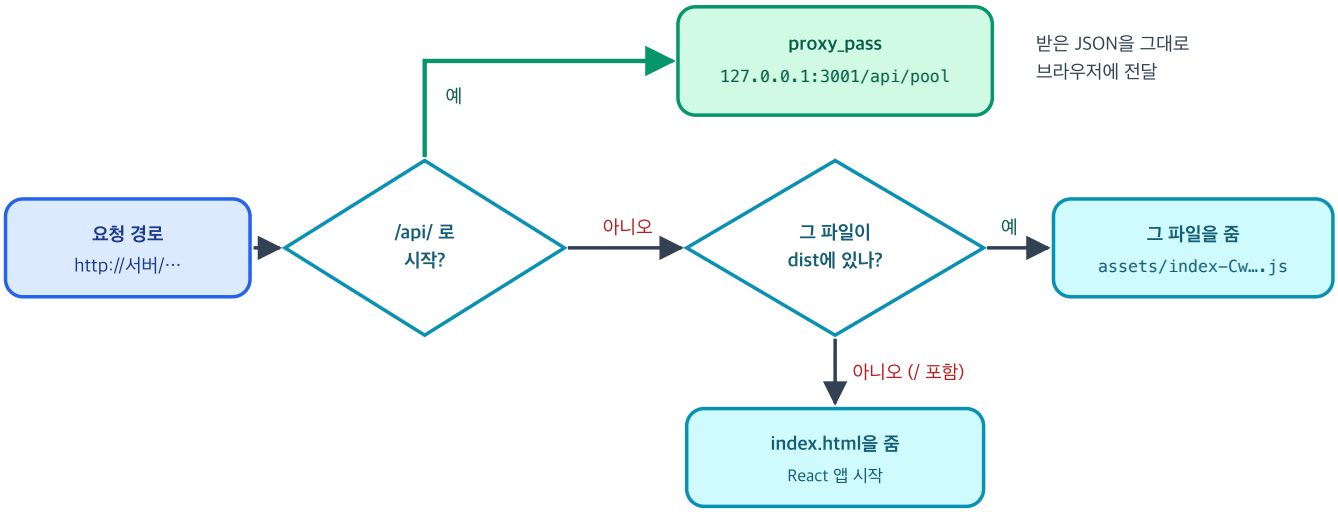
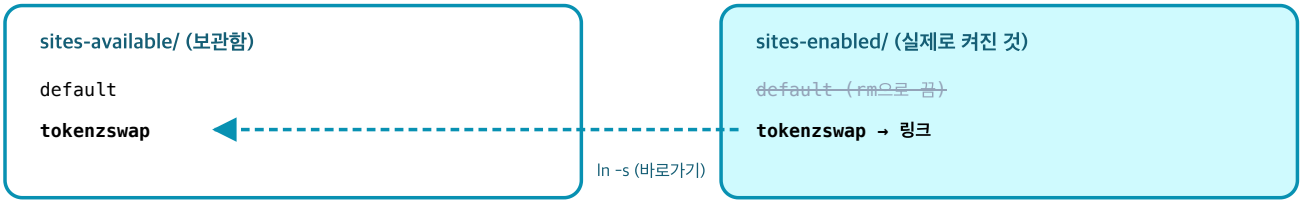


그림 13. location /api/ → proxy_pass, location / → try_files $uri $uri/ /index.html

```
# /etc/nginx/sites-available/tokenzswap  (저장소: deployment/server/nginx-tokenzswap.conf)
server {
    listen 80;                              # http
    server_name _;                          # 어떤 주소로 오든 (도메인이 생기면 여기에)
    root /var/www/tokenzswap;               # 화면 파일 폴더
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:3001;   # 뒤에 경로를 안 쓰면 /api/pool 그대로 붙여 넘김
    }
    location / {
        try_files $uri $uri/ /index.html;   # 파일 → 폴더 → 없으면 index.html
    }
}
```

sites-available과 sites-enabled




```
sudo ln -s /etc/nginx/sites-available/tokenzswap /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default      # 기본 환영 페이지 끄기 (링크만 지움)
sudo nginx -t                                 # 문법 검사: syntax is ok / test is successful
sudo systemctl reload nginx                   # 끊김 없이 새 설정 적용
```

> `nginx -t` 없이 reload하면, 설정에 오타가 있을 때 사이트가 통째로 내려갈 수 있습니다. 항상 `-t` 먼저.

왜 화면 파일을 `/home/ubuntu`가 아니라 `/var/www`에 두나: nginx는 `www-data` 계정으로 돌고, Ubuntu의 홈 폴더는 다른 계정이 못 들어가게 잠겨 있습니다. `/var/www`는 웹 파일의 표준 위치이고, `chown ubuntu`로 rsync가 쓸 수 있게 했습니다.

14. 방화벽과 fail2ban

> ufw는 "22, 80, 443번만 들어올 수 있다"를 적은 규칙이고, fail2ban은 "비밀번호를 여러 번 틀린 IP를 일정 시간 차단하는 프로그램"입니다.

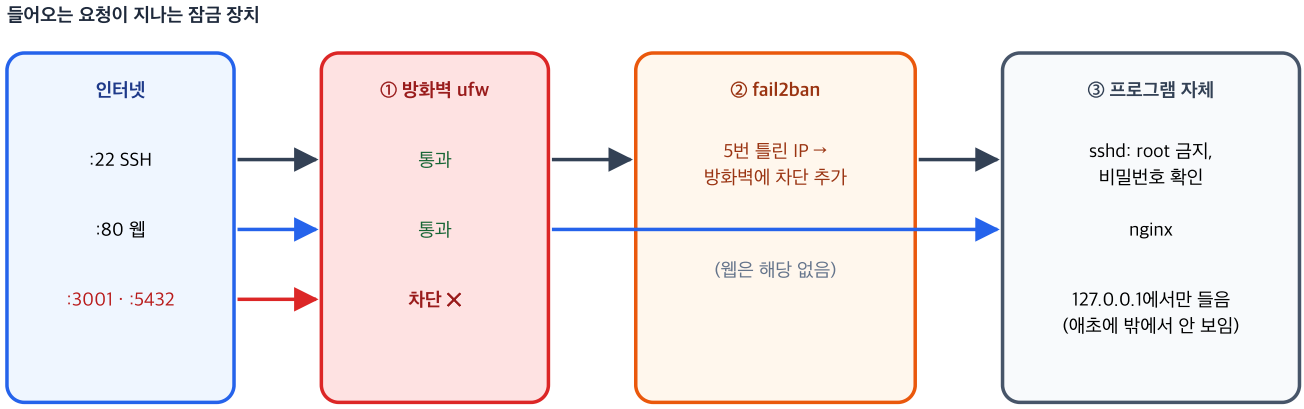


그림 14. 잠금이 여러 겹이라, 하나를 실수해도 다음 것이 막아 줍니다.

```
sudo ufw allow OpenSSH          # 22  ← 반드시 enable보다 먼저! (아니면 지금 연결이 끊기고 못 들어옴)
sudo ufw allow 'Nginx Full'     # 80, 443
sudo ufw enable
sudo ufw status

echo 'PermitRootLogin no' | sudo tee /etc/ssh/sshd_config.d/10-hardening.conf
sudo sshd -t && sudo systemctl reload ssh

sudo apt install -y fail2ban && sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

설치 몇 분 만에 본 실제 숫자

항목	값	뜻
Total failed	**546**	로그에 남은 SSH 비밀번호 실패. 거의 전부 봇
Currently banned	**3**	`109.160.32.x` 세 개: 같은 대역에서 나눠 시도하던 봇

공인 IP를 가진 서버는 켜자마자 항상 이런 시도를 받습니다. 비밀번호 로그인을 쓰는 동안은 **길고 무작위인 비밀번호**(`passwd`로 변경)와 fail2ban이 핵심 방어입니다. 나도 5번 틀리면 10분 동안 막힌다는 점만 기억하세요.

파일 이름이 `10-`인 이유: sshd는 `sshd_config.d/`의 파일을 이름 순으로 읽고 **먼저 나온 값**을 씁니다. 클라우드 이미지의 `50-cloud-init.conf`보다 앞서야 우리 값이 이깁니다.

15. 업데이트 흐름 (자세히)

코드를 고친 뒤 서버에 반영하는 과정입니다. **화면(프론트)**과 **캐시 서버**는 반영 방법이 다릅니다. 화면은 파일만 바꾸면 되고, 캐시 서버는 실행 중인 프로그램을 다시 켜야 합니다.

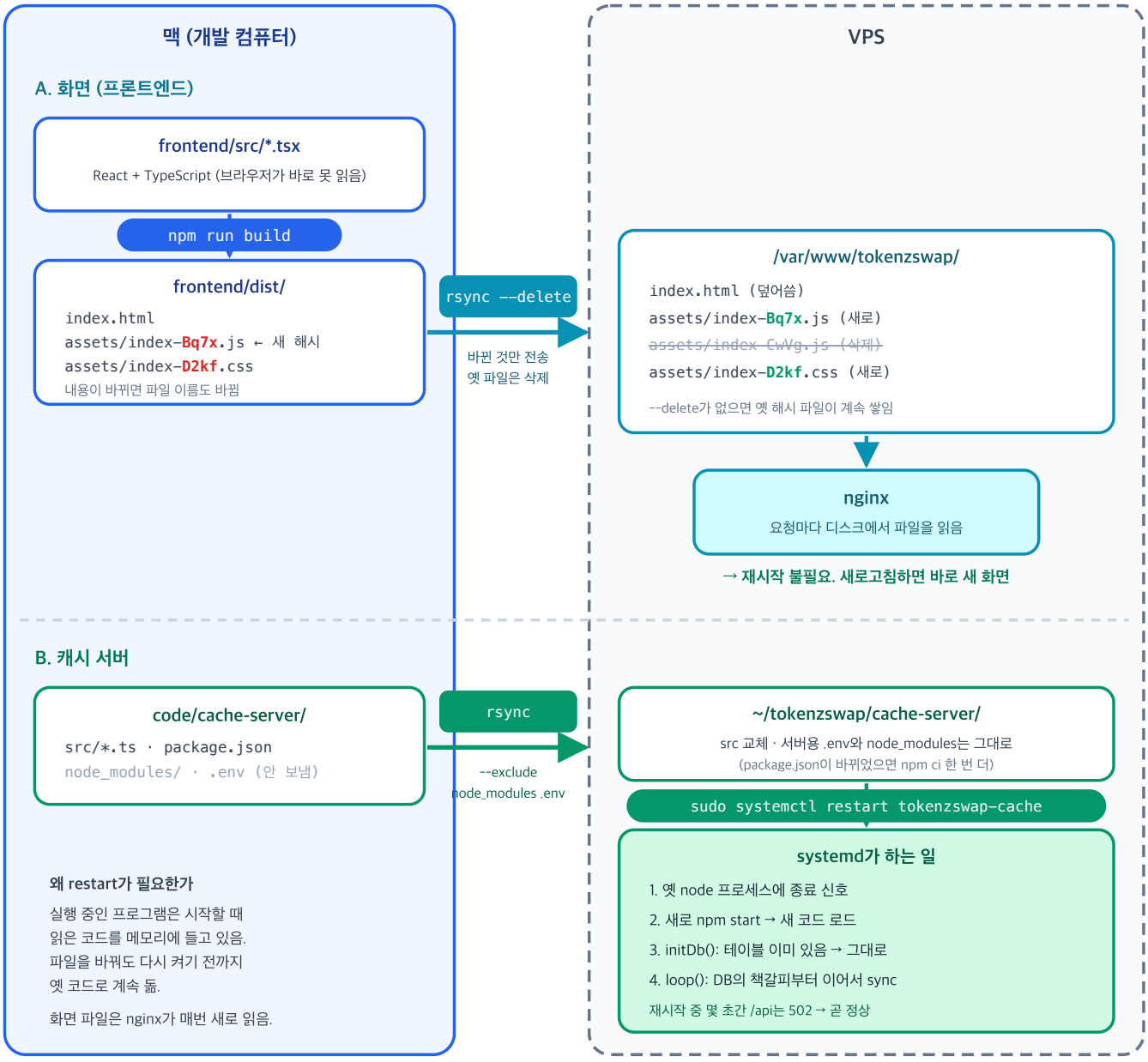


그림 15. 같은 rsync지만, 화면은 "파일 교체 = 끝", 캐시 서버는 "파일 교체 + 재시작"입니다.

15-2. 업데이트 명령 모음

화면만 바뀌었을 때

```
# 맥, 프로젝트 루트
cd frontend && npm run build && cd ..
rsync -av --delete frontend/dist/ ubuntu@서버:/var/www/tokenzswap/
# 끝. 브라우저 새로고침
```

캐시 서버 코드가 바뀌었을 때

```
# 맥
rsync -av --exclude node_modules --exclude .env code/cache-server/ ubuntu@서버:~/tokenzswap/cache-server/
# VPS
cd ~/tokenzswap/cache-server && npm ci          # package.json이 바뀐 경우만
sudo systemctl restart tokenzswap-cache
journalctl -u tokenzswap-cache -n 20 --no-pager  # "API on http://127.0.0.1:3001" 확인
```

설정 파일이 바뀌었을 때

바뀐 것	반영
nginx 설정	`sudo nginx -t && sudo systemctl reload nginx`
.service 파일	`sudo systemctl daemon-reload && sudo systemctl restart tokenzswap-cache`
VPS의 .env	`sudo systemctl restart tokenzswap-cache` (시작할 때만 읽음)
sshd 설정	`sudo sshd -t && sudo systemctl reload ssh` (기존 창은 열어 둔 채로 새 창에서 확인)

"반영이 안 된 것 같아요" 체크

증상	확인
화면이 옛날 그대로	강력 새로고침(⌘⇧R). `ls /var/www/tokenzswap/assets`에 새 해시 파일이 있나
/api가 502 Bad Gateway	nginx는 살았는데 3001이 응답 없음 → `systemctl status tokenzswap-cache`, `journalctl`
/api가 500 internal error	캐시 서버는 살았는데 DB 에러 → .env 비밀번호/포트(5432), `pg_isready`
숫자가 안 바뀜	로그에 `sync … rows`가 찍히나, `sync failed`가 있나 (Studio 접속 문제)

16. 문제 해결 사례와 체크리스트

이번에 실제로 만난 문제들

증상	원인	해결
`password authentication failed`	"비밀번호" 자리표시 글자를 그대로 입력	`\password tokenzswap`로 새로 정함
bash에서 `select …` 문법 에러	psql이 아니라 bash에 SQL 입력	`psql …`로 들어간 뒤 입력
`Unexpected character '/'`	GraphQL 문자열 안의 `//` 주석	백틱 안 주석 삭제
INSERT가 NOT NULL 위반	`row[c]` (DB 이름으로 서브그래프 값을 찾음)	`row[f]`
필터 `id: $cursor`로 항상 0개	"같다" 비교, 첫 cursor는 `''`	`id_gt`
`npm run dev`가 DB에 못 붙음	스크립트에 `--env-file=.env` 없음	package.json 스크립트에 추가
콘솔에 tbody whitespace 경고	`</tr> {/* 주석 */}`의 공백이 글자로 들어감	JSX 주석은 태그 안에 넣거나 삭제
Explorer가 HTTP 500	SSH 터널이 끊김 (하루 지남)	터널 다시 열기, `ServerAliveInterval=30`
ssh `Operation timed out`	IP 주소 틀림	관리 콘솔에서 IP 확인
터널이 맥에 안 생김	VPS 안에서 실행 / `-L` 없이 로그인만	맥에서 `ssh -N -L …`
rsync `mkdir … failed`	부모 폴더 없음	`mkdir -p` 먼저
`curl`만 치니 도움말	주소를 안 씀	`curl -s localhost:3001/api/pool; echo`

배포 체크리스트

확인 명령	기대 결과
`sudo ss -ltnp | grep -E '5432|3001'`	둘 다 `127.0.0.1`
`systemctl status tokenzswap-cache`	`active (running)`
`curl -s localhost/api/pool; echo` (VPS)	풀 JSON (nginx → 3001 → DB)
`curl -s http://서버/api/pool` (맥)	풀 JSON (밖에서도 됨)
`nc -z 서버 3001`, `nc -z 서버 5432` (맥)	둘 다 닫힘
`sudo ufw status`	OpenSSH, Nginx Full만 ALLOW
`sudo fail2ban-client status sshd`	jail 동작, banned 숫자 보임
VPS 재부팅 후 위 전부	그대로 (systemd 자동 시작)

한 장 요약

부품	한 문장
책갈피	지난번에 어느 블록까지 가져왔는지 적어 둔 숫자 하나 (sync_state.lastBlock)
_change_block	그 블록 이후에 생기거나 바뀐 엔티티만 달라는 서브그래프 필터
upsert	없으면 넣고 있으면 고치기 (ON CONFLICT … DO UPDATE)
$1	바깥 값을 SQL이 아니라 글자로만 다루게 하는 자리표시
127.0.0.1	이 컴퓨터 안에서만 접속 가능
SSH 터널	ssh를 친 컴퓨터에 입구를 만들어 서버 안으로 연결하는 개발용 통로
systemd	켜고, 죽으면 살리고, 부팅 때 켜 주는 관리자
nginx	/api는 캐시 서버로, 나머지는 화면 파일로 나누는 입구
ufw / fail2ban	문을 22·80·443만 열기 / 틀리는 IP 차단
업데이트	화면은 build + rsync, 캐시 서버는 rsync + restart